

Uniwersytet Wrocławski
Wydział Fizyki i Astronomii

Oliwer Urbaniak

System zarządzania pracą drukarki 3D w laboratorium
studenckim.

Praca inżynierska na kierunku
Informatyka Stosowana i Systemy Pomiarowe

Opiekun
dr Bartosz Brzostowski

Wrocław, 17 maja 2026

Spis treści

1	Wprowadzenie	6
1.1	Motywacja i cel pracy	6
1.2	Zakres pracy	6
1.3	Struktura pracy	6
2	Analiza wymagań i przegląd rozwiązań	7
2.1	Wymagania funkcjonalne	7
2.2	Wymagania niefunkcjonalne	7
2.3	Przegląd istniejących rozwiązań	9
2.3.1	Octoprint	9
2.3.2	Mainsail i Fluidt	9
2.3.3	Rozwiązania komercyjne	9
2.3.4	Wnioski	9
3	Wybór technologii	10
3.1	Języki programowania	10
3.1.1	TypeScript / Node.js	10
3.1.2	Python	10
3.2	Framework webowy - Express.js	10
3.3	Baza danych - Azure Cosmos DB	10
3.4	Przechowywanie plików - Azure Blob Storage	10
3.5	Kolejkowanie - Azure Service Bus	10
3.6	Sterowanie urządzeniem - Azure IoT Hub	10
3.7	Frontend - React i Vite	10
3.8	Kontenery - Docker i Docker Compose	10
3.9	CAD - SolidWorks	10
3.10	Porównanie wybranych alternatyw	10
4	Architektura systemu	11
4.1	Ogólna struktura	11
4.2	Diagram architektury	11
4.3	Model danych	11
4.3.1	Użytkownik (kontener <code>users</code>)	11
4.3.2	Zadanie (kontener <code>jobs</code>)	11
4.3.3	Refresh Token (kontener <code>refresh-token</code>)	11
4.4	Cykl życia zadania do wydruku	11
4.5	Bezpieczeństwo i uwierzytelnianie	11
5	Implementacja serwisów backendowych	12
5.1	Auth Service	12
5.1.1	Opis ogólny	12
5.1.2	Endpointy API	12
5.1.3	Kluczowe fragmenty implementacji	12
5.1.4	Weryfikacja e-mail	12
5.2	User Service	12
5.2.1	Opis ogólny	12

5.2.2	Endpointy API	12
5.2.3	Mechanizm autoryzacji	12
5.3	Files Service	12
5.3.1	Opis ogólny	12
5.3.2	Przepływ przesyłania pliku	12
5.4	Queue Service	12
5.4.1	Opis ogólny	12
5.4.2	Listener kolejki	12
5.4.3	Endpointy HTTP API	12
5.5	Printer Service	12
5.5.1	Opis ogólny	12
5.5.2	Harmonogram	12
5.5.3	Monitor gotowości	12
5.5.4	Listener telemetrii	12
5.5.5	Endpointy HTTP API	12
5.6	Video Service	12
6	Implementacja frontendu	13
6.1	Struktura aplikacji	13
6.2	Klient API	13
6.3	Routing i ochrona tras	13
6.4	Ekrany aplikacji	13
6.4.1	Strona główna	13
6.4.2	Dashboard użytkownika	13
6.4.3	Upload i planowanie	13
6.4.4	Kontrola druku	13
6.4.5	Panel administracyjny	13
6.5	Internacjonalizacja	13
6.6	Obsługa motywu	13
7	Agent Raspberry Pi - AddiPi Agent	14
7.1	Opis ogólny	14
7.2	Architektura agenta	14
7.3	Połączenie z IoT Hub	14
7.4	Obsługiwane metody bezpośrednie	14
7.5	Przebieg obsługi metody startPrint	14
7.6	Telemetria	14
7.7	Komunikacja z OctoPrint	14
8	Obudowa urządzenia - projekt CAD	15
8.1	Cel i zakres projektu mechanicznego	15
8.2	Moduł CASE - obudowa Raspberry Pi z kamerą	15
8.2.1	Podstawa projektowa	15
8.2.2	Zakres modyfikacji	15
8.2.3	Mocowanie kamery	15
8.2.4	Struktura plików CASE	15
8.3	Moduł STAND - podstawka regulowana	15
8.4	Złożenie całości	15

8.5	Parametry wydruku	15
8.6	Instrukcja montażu	15
9	Konfiguracja Raspberry Pi	16
9.1	Opis środowiska	16
9.2	Instalacja agenta	16
9.3	Automatyczne uruchamianie agenta przez systemd	16
9.4	Tunel Cloudflare - ekspozycja kamery	16
9.4.1	Instalacja cloudflared	16
9.4.2	Skrypt publikowania URL kamery	16
9.4.3	Cykliczne odnawianie URL	16
9.5	Konfiguracja crontab	16
9.5.1	Uzasadnienie parametrów	16
9.6	Problem z dostępnością sieci przy starcie	16
9.7	Podsumowanie konfiguracji Raspberry Pi	16
10	Wdrożenie i infrastruktura	17
10.1	Infrastruktura chmurowa	17
10.2	Inicjalizacja infrastruktury	17
10.3	Konteneryzacja - Dockerfiles	17
10.4	Docker Compose	17
10.5	Zmienne środowiskowe	17
10.6	Wdrożenie frontendu	17
10.7	Wdrożenie agenta	17
11	Testowanie systemu	18
11.1	Metodologia testowania	18
11.2	Testy jednostkowe - Auth Service	18
11.3	Testy integracyjne przez Postman	18
11.4	Test end-to-end - pełny przepływ druku	18
11.5	Testy wydajnościowe i obserwacje	18
12	Podsumowanie	19
12.1	Osiągnięcia	19
12.2	Napotkane trudności	19
12.3	Możliwości rozwoju	19
12.4	Wnioski	19
	Appendices	21
A	Konfiguracja środowiska deweloperskiego	21
A.1	Wymagania	21
A.2	Pierwsze uruchomienie	21
B	Struktura repozytoriów	21
C	Endpointy API - podsumowanie	21

Streszczenie

Niniejsza praca inżynierska opisuje projekt, implementację oraz wykonanie systemu *AddiPi* - rozproszonego systemu zarządzania pracą drukarki 3D przeznaczonego dla laboratorium studenckiego. System umożliwia użytkownikom przesyłanie plików G-code, planowanie oraz monitorowanie zadań do wydruku, a administratorom - pełną kontrolę nad kolejką i użytkownikami.

Architektura systemu oparta jest na podejściu mikroserwisowym. Wyodrębniono sześć niezależnych serwisów backendowych: uwierzytelniania (Auth Service), zarządzania użytkownikami (User Service), obsługi plików (Files Service), kolejkowania zadań (Queue Service), sterowania drukarką (Printer Service) oraz agenta działającego na urządzeniu Raspberry Pi (AddiPi Agent). Całość uzupełnia aplikacja frontendowa zbudowana w React oraz serwis wideo do podglądu kamery.

Komunikacja między serwisami odbywa się za pośrednictwem protokołu HTTP/REST oraz chmurowej kolejki Azure Service Bus. Dane przechowywane są w Azure Cosmos DB, a pliki G-code w Azure Blob Storage. Sterowanie drukarką realizowane jest przez Azure IoT Hub przy użyciu bezpośrednich wywołań metod (Direct Methods) na Raspberry Pi, na którym działa oprogramowane OctoPrint.

System zrealizowano przy użyciu języków TypeScript (Node.js) i Python. Wdrożenie odbywa się w kontenerach Docker, koordynowanych plikiem Docker Compose. Interfejs użytkownika obsługuje uwierzytelnianie tokenem JWT, przesyłanie plików, planowanie zadań, podgląd kamery w czasie rzeczywistym oraz panel administracyjny.

Słowa kluczowe: mikroserwisy, drukarka 3D, OctoPrint, Raspberry Pi, Azure Iot Hub, Azure Cosmos DB, Docker, TypeScript, Python, REST API, kolejkowanie zadań

1 Wprowadzenie

1.1 Motywacja i cel pracy

Drukarki 3D stanowią obecnie standardowe wyposażenie laboratoriów akademickich, warsztatów studenckich oraz zakładów produkcyjnych zorientowanych na wytwarzanie przyrostowe. Urządzenia te, pomimo rosnącej dostępności i malejących cen, pozostają zasobem współdzielonym - ich liczba jest zwykle ograniczona, a czas drukowania pojedynczego modelu może wynosić od kilkunastu minut do wielu godzin. W środowisku akademickim oraz przemysłowym rodzi to naturalne problemy organizacyjne: konieczność ręcznego umawiania się na czas druku, brak przejrzystości stanu kolejki oraz niemożność zdalnego monitorowania postępu pracy.

Celem niniejszej pracy jest zaprojektowanie i implementacja systemu *AddiPi*, który automatyzuje proces zarządzania drukarką 3D w laboratorium studenckim. System pozwala użytkownikom na:

- przysyłanie plików G-code przez przeglądarkę internetową,
- planowanie druków na określony termin lub dodawanie ich do kolejki,
- śledzenie postępu druku w czasie rzeczywistym, wraz z podglądem kamery,
- otrzymywanie powiadomień o zakończeniu lub błędzie druku.

Administratorzy laboratorium zyskują natomiast narzędzia do zarządzania użytkownikami, przeglądania historii wszystkich zadań oraz sterowania kolejką i urządzeniami.

1.2 Zakres pracy

Praca obejmuje:

1. analizę wymagań systemu i wybór technologii,
2. projekt architektury mikroservisowej,
3. implementację wszystkich komponentów backendowych i frontendowych,
4. konfigurację infrastruktury chmurowej (Microsoft Azure),
5. wdrożenie systemu przy użyciu kontenerów Docker,
6. testy poprawności działania.

1.3 Struktura pracy

Rozdział 2 zawiera analizę wymagań i przegląd istniejących rozwiązań. Rozdział 3 omawia wybrane technologie i uzasadnia decyzje projektowe. Rozdział 4 opisuje architekturę systemu. Rozdziały 5–7 szczegółowo omawiają implementację poszczególnych komponentów programowych. Rozdział 8 opisuje projekt mechaniczny obudowy w SolidWorks. Rozdział 9 omawia konfigurację Raspberry Pi, automatyzację przez cron i tunel Cloudflare. Rozdział 10 dotyczy wdrożenia i infrastruktury chmurowej Azure. Rozdział 11 przedstawia wyniki testów. Praca kończy się podsumowaniem w rozdziale 12.

2 Analiza wymagań i przegląd rozwiązań

2.1 Wymagania funkcjonalne

Na podstawie analizy potrzeb laboratorium studenckiego sformułowano następujące wymagania funkcjonalne:

Dla użytkowników:

- Rejestracja z weryfikacją adresu e-mail (ograniczoną do domeny uczelni @uwr.edu.pl).
- Logowanie z użyciem tokenów JWT (access token + refresh token).
- Przesyłanie plików G-code (maksymalnie 50 MB, format .gcode).
- Planowanie druku na wybrany termin lub dodanie do kolejki ogólnej.
- Przeglądanie własnych zadań z filtrowaniem według statusu.
- Podgląd postępu druku w czasie rzeczywistym (pasek postępu, temperatura dyszy i podłoża, obraz z kamery).
- Anulowanie i ponowne uruchamianie własnych zadań.

Dla administratorów:

- Pogląd i zarządzanie wszystkimi użytkownikami systemu.
- Zmiana roli użytkownika (użytkownik/administrator).
- Podgląd i zarządzanie wszystkimi zadaniami druku.
- Potwierdzenie gotowości drukarki do kolejnego zlecenia.
- Dostęp do metryk systemowych (liczba zadań według statusu). zawartość...

2.2 Wymagania niefunkcjonalne

- **Skalowalność** - architektura powinna umożliwiać dodanie kolejnych drukarek bez przebudowy systemu.
- **Niezawodność** - utrata wiadomości z kolejki nie powinna powodować trwałej utraty zadania.
- **Bezpieczeństwo** - dostęp do API chroniony tokenami JWT, hasła przechowywane w formie szyfru bcrypt.
- **Responsywność interfejsu** - aplikacja frontendowa powinna działać zarówno na urządzeniach desktopowych, jak i mobilnych.
- **Czas odpowiedzi** - operacje API powinny odpowiadać w czasie poniżej 2 sekund w warunkach normalnego obciążenia.
- **Przenośność** - wszystkie komponenty opakowane w kontenery Docker, uruchamiane jednym poleceniem.

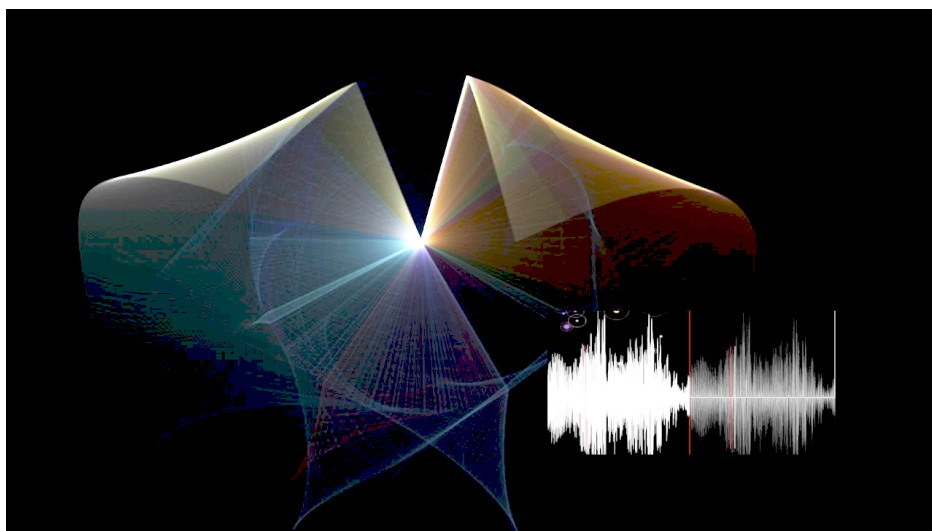
W swojej pracy możesz odnosić się do równań np. do równania (1) lub do rysunków, zarówno wektorowych, patrz rys. (1) jak i bitmapowych (2). Możesz też z powodzeniem cytować literaturę pojedynczo [3] lub kilka pozycji na raz [?, 1]. Jeśli chcesz to robić wygodniej załóż tzw. plik bibtex, ale możesz też pracować tak, jak w tym dokumencie.

$$\rho \left(\frac{\partial \vec{v}}{\partial t} + (\vec{v} \cdot \nabla) \vec{v} \right) = \rho \vec{f} - \nabla p + \mu \Delta \vec{v}, \quad (1)$$

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.



Rysunek 1: Podpis rysunku, który jest obrazem wektorowym (EPS).



Rysunek 2: To jest rysunek drugi, kilkadziesiąt wahadeł podwójnych z syntezą dźwięku.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure

dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. .. in the figure 2.2.

2.3 Przegląd istniejących rozwiązań

2.3.1 Octoprint

OctoPrint [2] jest najpopularniejszym oprogramowaniem do zarządzania drukarkami 3D. Działa jako serwer HTTP na Raspberry Pi i udostępnia interfejs webowy, API REST oraz system wtyczek. Umożliwia zdalne sterowanie drukarką, podgląd z kamery i monitorowanie postępu.

OctoPrint jest jednak przeznaczony dla jednego użytkownika zarządzającego jedną drukarką. Nie oferuje mechanizmu kolejkowania zadań wielu użytkowników, systemu uwierzytelniania opartego na rolach ani integracji z chmurą. W systemie AddiPi OctoPrint pełni rolę lokalnego sterownika drukarki, nad którym nadbudowano opisywaną w tej pracy warstwę zarządzania.

2.3.2 Mainsail i Fluidt

Mainsail i Fluidt to nowoczesne interfejsy webowe dla firmware Klipper. Podobnie jak OctoPrint, skierowane są do jednego użytkownika i nie przewidują obsługi kolejki wielu użytkowników.

2.3.3 Rozwiązania komercyjne

Oblio, Polar Cloud i Ultimaker Digital Factory to komercyjne platformy zarządzania flotą drukarek 3D w środowisku przemysłowym i edukacyjnym. Oferują zaawansowane funkcje, jednak wiążą się z kosztami subskrypcji i nie pozwalają na pełną kontrolę nad infrastrukturą - co w warunkach laboratorium akademickiego jest istotną przeszkodą.

2.3.4 Wnioski

Żadne z dostępnych rozwiązań o otwartym kodzie źródłowym nie łączy w sobie: systemu uwierzytelniania obsługującego wielu użytkowników, kolejkowania zadań z planowaniem czasowym, integracji z chmurą obliczeniową i sterowaniem przez IoT. System AddiPi wypełnia tę lukę, pozostając jednocześnie w pełni samodziśalający i wdrażany lokalnie.

3 Wybór technologii

3.1 Języki programowania

3.1.1 TypeScript / Node.js

3.1.2 Python

3.2 Framework webowy - Express.js

3.3 Baza danych - Azure Cosmos DB

3.4 Przechowywanie plików - Azure Blob Storage

3.5 Kolejowanie - Azure Service Bus

3.6 Sterowanie urządzeniem - Azure IoT Hub

3.7 Frontend - React i Vite

3.8 Kontenery - Docker i Docker Compose

3.9 CAD - SolidWorks

3.10 Porównanie wybranych alternatyw

4 Architektura systemu

4.1 Ogólna struktura

4.2 Diagram architektury

4.3 Model danych

4.3.1 Użytkownik (kontener users)

4.3.2 Zadanie (kontener jobs)

4.3.3 Refresh Token (kontener refresh-token)

4.4 Cykl życia zadania do wydruku

4.5 Bezpieczeństwo i uwierzytelnianie

5 Implementacja serwisów backendowych

5.1 Auth Service

5.1.1 Opis ogólny

5.1.2 Endpointy API

5.1.3 Kluczowe fragmenty implementacji

5.1.4 Weryfikacja e-mail

5.2 User Service

5.2.1 Opis ogólny

5.2.2 Endpointy API

5.2.3 Mechanizm autoryzacji

5.3 Files Service

5.3.1 Opis ogólny

5.3.2 Przepływ przesyłania pliku

5.4 Queue Service

5.4.1 Opis ogólny

5.4.2 Listener kolejki

5.4.3 Endpointy HTTP API

5.5 Printer Service

5.5.1 Opis ogólny

5.5.2 Harmonogram

5.5.3 Monitor gotowości

5.5.4 Listener telemetrii

5.5.5 Endpointy HTTP API

5.6 Video Service

6 Implementacja frontendu

6.1 Struktura aplikacji

6.2 Klient API

6.3 Routing i ochrona tras

6.4 Ekrany aplikacji

6.4.1 Strona główna

6.4.2 Dashboard użytkownika

6.4.3 Upload i planowanie

6.4.4 Kontrola druku

6.4.5 Panel administracyjny

6.5 Internacjonalizacja

6.6 Obsługa motywu

7 Agent Raspberry Pi - AddiPi Agent

7.1 Opis ogólny

7.2 Architektura agenta

7.3 Połączenie z IoT Hub

7.4 Obsługiwane metody bezpośrednio

7.5 Przebieg obsługi metody startPrint

7.6 Telemetria

7.7 Komunikacja z OctoPrint

8 Obudowa urządzenia - projekt CAD

8.1 Cel i zakres projektu mechanicznego

8.2 Moduł CASE - obudowa Raspberry Pi z kamerą

8.2.1 Podstawa projektowa

8.2.2 Zakres modyfikacji

8.2.3 Mocowanie kamery

8.2.4 Struktura plików CASE

8.3 Moduł STAND - podstawka regulowana

8.4 Złożenie całości

8.5 Parametry wydruku

8.6 Instrukcja montażu

9 Konfiguracja Raspberry Pi

9.1 Opis środowiska

9.2 Instalacja agenta

9.3 Automatyczne uruchamianie agenta przez systemd

9.4 Tunel Cloudflare - ekspozycja kamery

9.4.1 Instalacja cloudflared

9.4.2 Skrypt publikowania URL kamery

9.4.3 Cykliczne odnawianie URL

9.5 Konfiguracja crontab

9.5.1 Uzasadnienie parametrów

9.6 Problem z dostępnością sieci przy starcie

9.7 Podsumowanie konfiguracji Raspberry Pi

10 Wdrożenie i infrastruktura

10.1 Infrastruktura chmurowa

10.2 Inicjalizacja infrastruktury

10.3 Konteneryzacja - Dockerfiles

10.4 Docker Compose

10.5 Zmienne środowiskowe

10.6 Wdrożenie frontendu

10.7 Wdrożenie agenta

11 Testowanie systemu

11.1 Metodologia testowania

11.2 Testy jednostkowe - Auth Service

11.3 Testy integracyjne przez Postman

11.4 Test end-to-end - pełny przepływ druku

11.5 Testy wydajnościowe i obserwacje

12 Podsumowanie

12.1 Osiągnięcia

12.2 Napotkane trudności

12.3 Możliwości rozwoju

12.4 Wnioski

Literatura

- [1] Alexandre Joel Chorin. Numerical solution of the navier-stokes equations. *Mathematics of computation*, 22(104):745–762, 1968.
- [2] Gina Häußge. Octoprint — The snappy web interface for your 3D printer. <https://octoprint.org>. Dostęp: 2025.
- [3] Maciej Matyka, Arzhang Khalili, and Zbigniew Koza. Tortuosity-porosity relation in porous media flow. *Physical Review E*, 78(2):026306, 2008.

Appendices

A Konfiguracja środowiska deweloperskiego

A.1 Wymagania

A.2 Pierwsze uruchomienie

B Struktura repozytoriów

C Endpointy API - podsumowanie